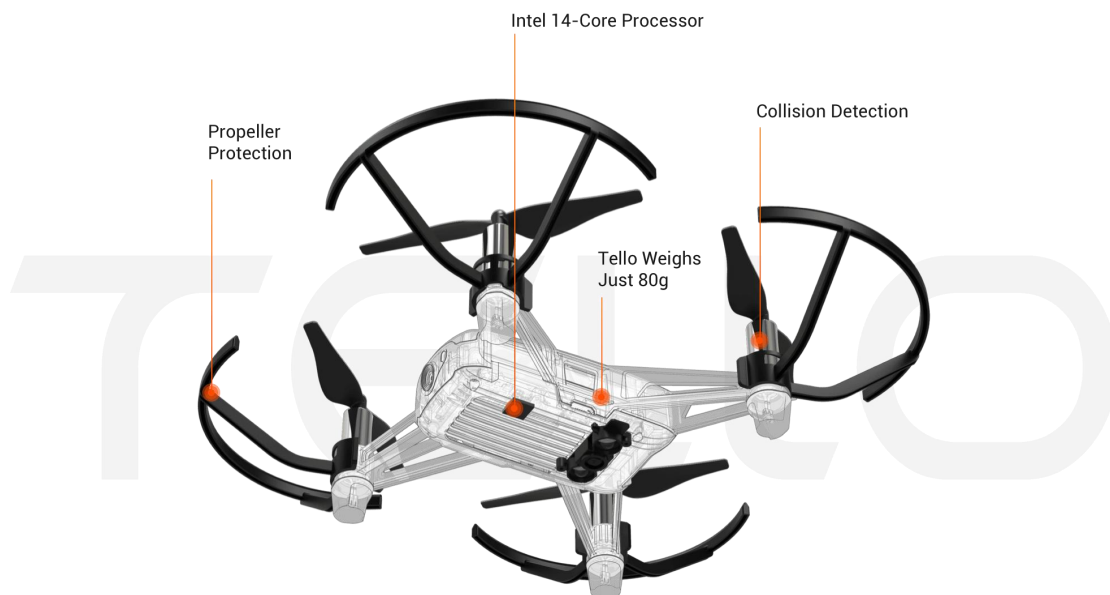


无人机子赛道参赛方案说明

1. 参赛飞行器简介

本文所使用的飞行器为 Tello 四旋翼无人机（如图 1 所示），Tello 是一款由 DJI（大疆创新）推出的小型智能无人机，它结合了先进的飞行技术、视觉传感器和便携设计。



1-1 外观介绍

Tello 具有轻巧、小巧的设计，重量约 80 克。其外壳采用高强度 ABS 塑料制成，具有一定的抗撞击能力。

1-2 飞行性能

Tello 配备了一套稳定的飞行控制系统，可以实现稳定的飞行、悬停和降落等基本飞行动作。在通常情况下，其电池续航一般在 12-3 分钟左右。

1-3 视觉系统

Tello 配备了一颗 5 百万像素的摄像头，可以拍摄 720p 的高清视频和 5MP 的照片。且支持实时图传，可以通过无人机上的 Wi-Fi 将实时视频传输到智能设备上，用户可以通过手机等设备实时观看飞行画面。

1-4 无人机控制

Tello 可以通过手机应用程序进行控制，适用于 iOS 或 Android 系统。也可以通过可选的物理遥控器进行控制，以获取更精确的控制体验。此外，tello 具有良好的编程接口，允许用户使用 Python 等编程语言编写程序进行控制和自定义飞行，利用 Tello SDK，用户可编程控制 tello 实现各种自定义飞行动作与任务。

2. 项目算法建模及程序设计

2-1 项目一（穿环）模型的建立：

欲使无人机顺利穿过圆环，则应该尽可能使其沿圆环中心区域飞行，最好是能从圆环的正中心通过（理想状态），这样可在最大程度上避免无人机碰到圆环边界。如图 1，调用 OpenCV 库中的霍夫圆环检测方法，对圆环进行检测，计算出其圆心 O 在 tello 前置摄像头中的坐标 (x_1, y_1) ，对比 tello 摄像头中心点的坐标 $O_1(x_0, y_0)$ ，在考虑到 tello 飞行误差的前提下，对 tello 飞行方向进行纠正，尽量使摄像头中心点与圆环圆心重合，尽量使 tello 从圆环中心区域穿过。从而提高成功率。编写程序 1（见附录）实现 tello 自主穿环。

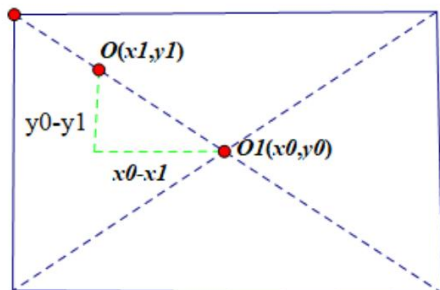


图 1

此外，由于圆环位置及大小已知，一种比较自然的想法是，在 tello 飞行误差允许的范围内，使其沿固定轨迹飞行，进而穿过圆环。

2-2 项目二（巡线）模型的建立：

欲使无人机依照指定曲线飞行，利用无人机的摄像头实时获取地面图像，利用图像处理算法凸显曲线，同时屏蔽掉其它无关事物及颜色的干扰，将无人机摄像头获取到的屏幕等分为三部分，通过计算并判断各子屏幕中曲线像素所占的面积比例是否达到设定的阈值，从而调节无人机的运动，使之依轨道航行。编写程序 2（见附录）实现 tello 自主巡线。

2-3 项目三（避障）模型的建立：

参赛所用的 tello 无人机，虽未配备测距模块，但可充分利用障碍物的颜色及形状已知这一条件，捕获 tello 前置摄像头的实时视频帧，利用计算机视觉库中的相关算法对每一帧图像进行掩模处理，并进行图像形态学运算，对除障碍物以外的环境事物进行屏蔽，从而简化周围环境，减少噪声的干扰。因为障碍物形状为矩形，对无人机与障碍物位置进行分析可知，在 tello 的摄像头中，以障碍物轮廓的几何中心相对于 tello 的前置摄像头视野的中心位置建立数学模型，障碍物相对于无人机的位置可以大致下两种情况：1) 障

障碍物在无人机左侧（此时，障碍物轮廓中心对的横坐标小于图像中心点的横坐标），此时，tello 可向右飞行避障；2）障碍物在无人机右侧（此时，障碍物轮廓中心的横坐标大于图像中心点的横坐标，此时，tello 可向左飞行避障。当摄像头视野中未检测到障碍物轮廓时，可向前飞行。此外，可对 tello 每一小段的前飞距离求和来判断是否已经穿越障碍区域。

$$\text{避障函数} = \begin{cases} \text{向右偏航} & , x_0 > x_1 \\ \text{向左偏航} & , x_0 \leq x_1 \\ \text{向前航行,} & \text{未检测到障碍物轮廓} \end{cases}$$

其中， x_0 表示摄像头视野中心点横坐标， x_1 表示障碍物轮廓的几何中心的横坐标。避障程序 3（见附录）。

2-4 项目四（自主降落）模型建立

无人机完成项目三后，通过键盘切换到项目四，由于降落标志具体方位未知，欲降落到指定区域，则必须首先通过无人机的镜头检测出降落标志，本文采用旋转分段寻找的方式，搜索降落标志，无人机通过镜头捕获到降落标志后，又将面临以下两个问题：

- 1) 降落标志不一定恰好会落在镜头中心区域。
- 2) 无人机所在位置移动到指定降落区域的水平位移未知。

针对问题 1) 本文采用区间逼近的算法来实现降落标志向镜头中心纠正，即横坐标左右纠正（向镜头中心的横坐标靠拢）；纵坐标向下纠正（向镜头视野的下边界靠拢），从而使得无人机正对降落区域。

针对问题 2) 在问题 1) 中调整使得无人机正对降落区域后，如图 2 所示：

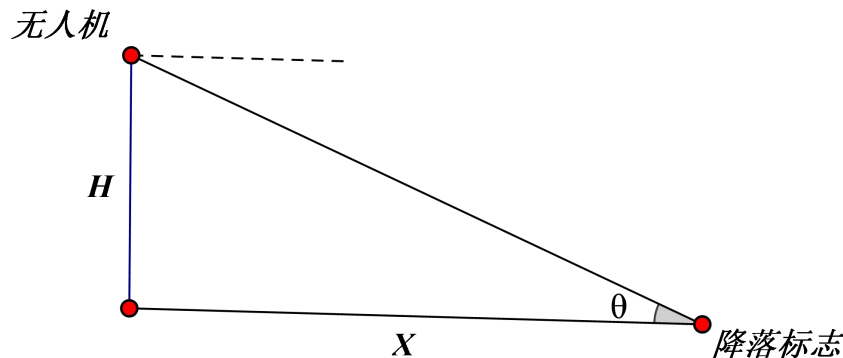


图 2（无人机与降落区域侧视图）

其中， H 表示无人机距地面的垂直高度， θ 表示降落标志与无人机镜头最大下视范围与地面的夹角， X 表示无人机到降落区域中心的水平位移。存在以下几何关系：

$$\tan \theta = \frac{H}{X}$$

通过测量标定可得到 θ 近似值，同时可通过 tello SDK 实时获取无人机的垂直高度 H

故，可推得

$$X = H / \tan \theta$$

编写程序 4（见附录）从而使 tello 完成自主降落。

附录

程序 1:

```
from djitellopy import Tello
import time
tello = Tello()
tello.connect()
def Through():
    ...
    项目一穿环函数
    ...
    tello.takeoff()
    time.sleep(0.3)
    tello .move_forward(120)
    tello.move_left(100)
    tello.move_forward(60)
    tello.move_right(100)
    tello.move_forward(60)
```

```
Through()
tello.land()
```

程序 2:

```
import cv2
import numpy as np
from djitellopy import Tello
import time
# Initialize Tello drone
me = Tello()
me.connect()
print("Battery:", me.get_battery())
me.takeoff()
time.sleep(0.5)
me.move_down(60)

print("Battery:", me.get_battery())

# Start video stream from the drone
me.streamon()

# HSV threshold values
hsvVals = [0, 0, 0, 179, 255, 74]
```

```

# Other constants and parameters
sensors = 3
threshold = 0.2
width, height = 480, 360
sensitivity = 3

def thresholding(img):
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    lower = np.array([hsvVals[0], hsvVals[1], hsvVals[2]])
    upper = np.array([hsvVals[3], hsvVals[4], hsvVals[5]])
    mask = cv2.inRange(hsv, lower, upper)
    return mask

def getCnotours(imgThres, img):
    contours, hierarchy = cv2.findContours(imgThres,
cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
    if contours:
        biggest = max(contours, key=cv2.contourArea)
        x, y, w, h = cv2.boundingRect(biggest)
        cx = x + w // 2
        cy = y + h // 2
        cv2.drawContours(img, [biggest], -1, (0, 255, 0), 7)
        return cx
    return None

def getSensorOutput(imgThres, sensors):
    imgs = np.hsplit(imgThres, sensors)
    totalPixels = img.shape[1] // sensors * img.shape[0]
    senOut = []
    for x, im in enumerate(imgs):
        pixelCount = cv2.countNonZero(im)
        if pixelCount > threshold * totalPixels:
            senOut.append(1)
        else:
            senOut.append(0)
        cv2.imshow(str(x), im)
    return senOut

def sendCommands(senOut, cx):
    lr = (cx - width // 2) / sensitivity
    lr = int(np.clip(lr, -10, 10))

```

```

if lr < 2 and lr > -2:
    lr = 0

if senOut == [1, 0, 0]:
    me.send_rc_control(-10, 10, 0, 0)
    print("1, 0, 0")
elif senOut == [1, 1, 0]:
    me.send_rc_control(-10, 10, 0, -30)
    print("1, 1, 0")
elif senOut == [0, 1, 0]:
    me.send_rc_control(0, 15, 0, 0)
    print("1, 1, 1")
elif senOut == [0, 0, 0]:
    me.send_rc_control(0, -10, 0, 20)
    print("0, 0, 0")
elif senOut == [0, 0, 1]:
    me.send_rc_control(10, 10, 0, 0)
    print("0, 0, 1")
elif senOut == [0, 1, 1]:
    me.send_rc_control(10, 15, 0, 30)
    print("0, 1, 1")
elif senOut == [1, 1, 1]:
    me.send_rc_control(0, 0, 0, -15)
time.sleep(0.5)

```

```

while True:
    img = me.get_frame_read().frame
    img = cv2.resize(img, (width, height))
    img = cv2.flip(img, 0)
    imgThres = thresholding(img)
    cx = getCnotours(imgThres, img)
    if cx is not None:
        senOut = getSensorOutput(imgThres, sensors)
        sendCommands(senOut, cx)

    cv2.imshow("Output", img)
    cv2.imshow("Path", imgThres)

    if cv2.waitKey(1) & 0xFF == ord(' '):
        # me.land()
        me.streamoff()
        cv2.destroyAllWindows()
        break

```

程序 3:

```
import cv2
import numpy as np
from djitellopy import Tello
import time

class PID:
    def __init__(self, Kp, Ki, Kd):
        self.Kp = Kp
        self.Ki = Ki
        self.Kd = Kd
        self.prev_error = 0
        self.integral = 0

    def compute(self, error):
        self.integral += error
        derivative = error - self.prev_error
        output = self.Kp * error + self.Ki * self.integral + self.Kd
* derivative
        self.prev_error = error
        return output

def detect_obstacles(frame):
    # 将图像转换为 HSV 色彩空间
    hsv_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    # 定义障碍物颜色的 HSV 范围
    lower_color = (7, 36, 238)
    upper_color = (27, 255, 255)

    # 根据颜色范围创建一个掩码
    mask = cv2.inRange(hsv_frame, lower_color, upper_color)

    # 对掩码进行形态学操作，以减小噪声
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)

    # 找到轮廓
```



```

    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

    return contours

def main():
    # 初始化 Tello 无人机
    tello = Tello()
    tello.connect()
    x = tello.get_battery()
    print(x)
    tello.takeoff()
    # 初始化 PID 控制器
    pid_controller = PID(Kp=0.5, Ki=0.01, Kd=0.1)

    # 启动视频流
    tello.streamon()
    foward = 0
    while foward<=400:
        # 读取帧
        frame = tello.get_frame_read().frame
        #cv2.imshow("frame_o", frame)
        # 检测障碍物轮廓
        obstacles = detect_obstacles(frame)
        if len(obstacles) > 0:
            # 计算最大轮廓
            max_contour = max(obstacles, key=cv2.contourArea)

            # 获取最大轮廓的外接矩形
            x, y, w, h = cv2.boundingRect(max_contour)

            # 计算障碍物中心点坐标
            obstacle_center_x = x + w // 2

            # 获取图像中心点坐标
            frame_center_x = frame.shape[1] // 2

            # 控制左右平移
            if obstacle_center_x < frame_center_x:
                print("障碍物在图像 left 侧，向 right 飞行!")
                pid_output = pid_controller.compute(frame_center_x -
obstacle_center_x)
                tello.send_rc_control(int(pid_output), 0,0, 0) #
右平移

```

```

        time.sleep(1.5)
        tello.send_rc_control(0,0,0,0)
    else:
        print("障碍物在图像 right 侧, 向 left 飞行!")
        pid_output =
pid_controller.compute(obstacle_center_x - frame_center_x)
        tello.send_rc_control(-int(pid_output), 0, 0,0) #
左平移

        time.sleep(1.5)
        tello.send_rc_control(0,0,0,0)
    else:#前方无障碍物, 前飞
        print("前方没有障碍物, 以 15cm/s 的速度前飞 2 秒, 前进 30cm
左右")

        tello.send_rc_control(0,15,0,0)
        time.sleep(2)
        tello.send_rc_control(0,0,0,0)
        foward += 30

    # 显示图像
    cv2.imshow("Obstacle Detection", frame)
    # time.sleep(0.06)

```

程序 4:

```

def detect_landmark(frame):
    #将图像转换为 HSV 色彩空间
    hsv_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

    # 定义降落标志颜色的 HSV 范围
    lower_color = (117,69,34)
    upper_color = (179,255,255)

    # 根据颜色范围创建一个掩码
    mask = cv2.inRange(hsv_frame, lower_color, upper_color)

    # 对掩码进行形态学操作, 以减小噪声
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)

    # 找到轮廓
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    valid_obstacles = []
    for contour in contours:

```

```

        if len(contour) >= 3: # 确保轮廓至少有 3 个点
            area = cv2.contourArea(contour)
            if area > 10: # 根据需要调整面积阈值（本例中为 100）
                valid_obstacles.append(contour)
    for valid in valid_obstacles:
        x, y, w, h = cv2.boundingRect(valid)
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
        obstacle_center_x = x + w // 2
        obstacle_center_y = y + h // 2
        cv2.circle(frame, (obstacle_center_x, obstacle_center_y), 5,
(0, 0, 255), -1)

    return valid_obstacles

tello = Tello()
tello.connect()
x = tello.get_battery()
print(x)

tello.takeoff()

H = tello.get_height()
print(f"当前高度: {H}")
#初始化 PID 控制器
pid_controller = PID(Kp=0.5, Ki=0.01, Kd=0.1)

# 启动视频流
tello.streamon()
sensitivity = 3
print("-----避障项目已完成, 进入降落环节! -----")
    #1 寻找降落标志
    print("----开始寻找降落标志----")
    while True:

        # 读取帧
        frame = tello.get_frame_read().frame
        # 检测障碍物轮廓
        landMark = detect_landmark(frame)
        cv2.imshow("LandMark_img", frame)
        if len(landMark) > 0:

            # 计算最大轮廓
            max_contour = max(landMark, key=cv2.contourArea)

```

```

# 获取最大轮廓的外接矩形
x, y, w, h = cv2.boundingRect(max_contour)

# 计算降落标志中心点坐标
land_center_x = x + w // 2
land_center_y = y + h // 2
print("降落标志数据: ")
print(f"[{land_center_x},{land_center_y}]")
# 获取图像中心点坐标
frame_center_x = frame.shape[1] // 2
frame_center_y = frame.shape[0] // 2
lr = ( land_center_x-frame_center_x ) //sensitivity
lr = int(np.clip(lr, -20, 20))#之前数据: (-15,15)
if lr < 2 and lr > -2:
    lr = 0

if abs(frame_center_x-land_center_x)>50:
    if land_center_x < frame_center_x :#降落标志在偏左的区域

        tello.send_rc_control(lr,0,0,0)
        time.sleep(0.4)
        tello.send_rc_control(0,0,0,0)
    else:#降落标志在偏右的区域

        tello.send_rc_control(lr,0,0,0)
        time.sleep(0.4)
        tello.send_rc_control(0,0,0,0)
else:
    if abs(land_center_y-frame.shape[0])>100:
        tello.send_rc_control(0,10,0,0)
        time.sleep(0.5)
        tello.send_rc_control(0,0,0,0)
    else:
        print(f"当前高度: {H}")
        print("-----OK, 开始降落-----")
        tello.move_forward(int (H/0.66))
        time.sleep(0.8)
        tello.land()
        break
else:
    print("没找到降落标志 ")
    tello.send_rc_control(0,0,0,-20)#
    time.sleep(0.5)

```

```
tello.send_rc_control(0,0,0,0)
print("frame.shape:", frame.shape)
tello.streamoff()
cv2.destroyAllWindows()
```